

## CURSO DE MLOPS

Tema 5 — Contenerización con Docker

# Caso Práctico: Servir el Modelo de Riesgo Crediticio como API con Docker + FastAPI

Adaptado para GitHub Codespaces

NIVEL  
Intermedio

PROYECTO  
Riesgo Crediticio

ENTORNO  
GitHub  
Codespaces

DURACIÓN  
90 - 120 min

■ **Todo el laboratorio se ejecuta desde la terminal integrada del Codespace** (Ctrl+` para abrirla). Docker ya está disponible en el entorno — no necesitas instalar nada en tu máquina local.  
**Prerrequisito:** `artifacts/modelo.pkl` debe existir (generado en el Tema 4 con `run_pipeline.py`).

## ¿Qué estamos haciendo y por qué?

El modelo existe como `artifacts/modelo.pkl`, pero sigue siendo inútil para los sistemas del banco: no hay forma de enviarle datos de un cliente nuevo y recibir una predicción.

Creamos una **API REST con FastAPI** que expone el modelo como servicio HTTP, la empaquetamos en una **imagen Docker** y la ejecutamos como contenedor. Cualquier sistema del banco puede llamar a `POST /predecir` y recibir el score de riesgo en menos de 100 ms.

## ¿Qué lograrás al final?

- Entender qué es Docker y cómo encaja en el ciclo MLOps
- Escribir una API REST con FastAPI que carga y sirve el `modelo.pkl`
- Crear un Dockerfile que empaqueta la API con todas sus dependencias
- Construir una imagen Docker y correrla como contenedor en el Codespace
- Testear manualmente los endpoints `/predecir`, `/health` y `/docs`
- Usar `docker compose` para levantar API + MLflow en un comando

## Arquitectura de la solución

```

Core bancario / App web / Analista / curl / Swagger UI
| HTTP POST /predecir {"Prct_uso_tc": 0.85, ...}
v
+-----+
| Docker Container (puerto 8000 -> reenviado) |
| FastAPI (uvicorn :8000) |
| POST /predecir GET /health GET /docs |
| modelo.pkl (RandomForestClassifier) |
+-----+
| {"score_riesgo":0.82,"decision":"RECHAZAR",...}
v
Sistema del banco toma la decisión

```

### PASO 1

## Actualizar requirements.txt

Archivo raíz

txt – requirements.txt

```

# MLOps Tema 5 – API Docker + FastAPI
# Pipeline (Temas 1-4)
pandas>=1.5
numpy>=1.23
scikit-learn>=1.2
imbalanced-learn>=0.10
matplotlib>=3.6
dvc>=3.0
mlflow>=2.0
scipy>=1.9
pytest>=7.0

# API – Tema 5
fastapi>=0.100
uvicorn[standard]>=0.22
pydantic>=2.0
httpx>=0.24
python-multipart>=0.0.6

```

bash

```

cd /workspaces/banco-wiesse-mlops
pip install -r requirements.txt

# Verificar que el modelo del Tema 4 existe
ls artifacts/modelo.pkl
# Si no existe, primero ejecutar: python run_pipeline.py

```

**PASO  
2****api/schemas.py — Validación de entrada y salida**

api/schemas.py

Pydantic valida automáticamente tipos y rangos. Si falla, FastAPI devuelve un HTTP 422 antes de que el modelo reciba datos corruptos:

```
bash
```

```
mkdir -p api
touch api/__init__.py
```

```
python - api/schemas.py
```

```
"""
api/schemas.py — Modelos Pydantic para validación de entrada y salida.
Pydantic valida automáticamente tipos y rangos.
Si falla -> HTTP 422 con mensaje claro, el modelo no recibe datos corruptos.
"""

from pydantic import BaseModel, Field
from typing import Literal


class ClienteInput(BaseModel):
    """Datos de un cliente para predecir probabilidad de default."""
    Prct_uso_tc: float = Field(..., ge=0.0, le=10.0,
                               description='Porcentaje uso tarjeta crédito')
    Prct_deuda_vs_ingresos: float = Field(..., ge=0.0, le=10.0,
                                          description='Ratio deuda / ingresos')
    Mto_ingreso_mensual: float = Field(..., gt=0.0,
                                       description='Ingreso mensual en USD')
    Nro_dependiente: float = Field(..., ge=0.0, le=20.0,
                                   description='Número de dependientes')
    Edad: int = Field(..., ge=18, le=100,
                     description='Edad del cliente')
    Nro_prestao_retrasados: int = Field(..., ge=0,
                                       description='Préstamos con retraso >3 meses')
    Nro_prod_financieros_deuda: int = Field(..., ge=0,
                                           description='Número de productos de deuda')
    Nro_retraso_60dias: int = Field(..., ge=0,
                                   description='Retrasos >60 días')
    Nro_creditos_hipotecarios: int = Field(..., ge=0,
                                           description='Créditos hipotecarios')
    Nro_retraso_ultm3anios: int = Field(..., ge=0,
                                       description='Retrasos últimos 3 años')

    model_config = {'json_schema_extra': {'example': {
        'Prct_uso_tc': 0.85,
        'Prct_deuda_vs_ingresos': 0.62,
        'Mto_ingreso_mensual': 1800.0,
        'Nro_dependiente': 2.0,
        'Edad': 32,
        'Nro_prestao_retrasados': 3,
        'Nro_prod_financieros_deuda': 8,
        'Nro_retraso_60dias': 1,
        'Nro_creditos_hipotecarios': 1,
        'Nro_retraso_ultm3anios': 2,
    }}}


class PrediccionOutput(BaseModel):
    """Respuesta de la API con el resultado de la predicción."""
    score_riesgo: float # probabilidad de default [0-1]
```

```

decision:          Literal['APROBAR', 'REVISAR', 'RECHAZAR']
nivel_riesgo:      Literal['BAJO', 'MODERADO', 'ALTO', 'MUY ALTO']
probabilidad_default: float                                # mismo que score_riesgo, más descriptivo
umbral_usado:      float                                # UMBRAL_RECHAZAR = 0.60
modelo:           str                                    # tipo de algoritmo usado

class HealthResponse(BaseModel):
    """Respuesta del endpoint de salud."""
    status: str
    modelo: str
    version: str
    recall_entrenamiento: float

```

PASO  
3

## api/predictor.py — Carga del modelo y predicción

api/predictor.py

El modelo se carga **una sola vez** al arrancar la API (evento *startup*). Las predicciones siguientes no vuelven a leer el disco:

```

python - api/predictor.py

"""
api/predictor.py — Carga el modelo.pkl y ejecuta predicciones.
El modelo se carga UNA sola vez al arrancar la API (startup event).
Las predicciones subsecuentes no vuelven a leer el disco.
"""

import pickle, json, logging
from pathlib import Path

import numpy as np
import pandas as pd

log = logging.getLogger(__name__)

# Rutas relativas al directorio del proyecto
MODEL_PATH = Path('artifacts/modelo.pkl')
METRICS_PATH = Path('artifacts/metrics.json')

# Umbrales de decisión
UMBRAL_RECHAZAR = 0.60 # score >= 0.60 -> RECHAZAR
UMBRAL_REVISAR = 0.40 # score >= 0.40 -> REVISAR

class Predictor:
    """Singleton que encapsula la lógica de predicción."""

    def __init__(self):
        self.modelo = None
        self.metrics = {}
        self.features_orden = []

    def cargar(self) -> None:
        """Carga modelo y metadatos desde disco. Llamar UNA vez en startup."""
        if not MODEL_PATH.exists():
            raise FileNotFoundError(
                f'Modelo no encontrado: {MODEL_PATH}. '
                f'Ejecuta primero: python run_pipeline.py'
            )

        with open(MODEL_PATH, 'rb') as f:

```

```

        self.modelo = pickle.load(f)

    if METRICS_PATH.exists():
        with open(METRICS_PATH) as f:
            self.metrics = json.load(f)

    # Orden de features del entrenamiento
    features_path = Path('artifacts/features.csv')
    if features_path.exists():
        df_feat = pd.read_csv(features_path, nrows=0)
        excluir = ['Default', 'ID']
        self.features_orden = [c for c in df_feat.columns if c not in excluir]

    log.info('Modelo cargado: %s', type(self.modelo).__name__)
    log.info('Features en orden: %s...', self.features_orden[:3])

def predecir(self, datos: dict) -> dict:
    """Recibe dict con los campos del cliente y retorna la predicción."""
    if self.modelo is None:
        raise RuntimeError('Modelo no cargado. Llama a cargar() primero.')

    # Calcular features derivadas
    datos['Score_retrasos'] = (
        datos.get('Nro_prestao_retrasados', 0) * 3 +
        datos.get('Nro_retraso_60dias', 0) * 5 +
        datos.get('Nro_retraso_ultim3anios', 0) * 2
    )

    # Construir DataFrame en el orden correcto
    if self.features_orden:
        X = pd.DataFrame([{'k': datos.get(k, 0.0) for k in self.features_orden}])
    else:
        X = pd.DataFrame([datos])

    # Score de riesgo (probabilidad de default)
    proba = float(self.modelo.predict_proba(X)[0][1])

    # Lógica de decisión basada en umbrales
    if proba >= UMBRAL_RECHAZAR:
        decision = 'RECHAZAR'
        nivel_riesgo = 'MUY ALTO' if proba >= 0.80 else 'ALTO'
    elif proba >= UMBRAL_REVISAR:
        decision = 'REVISAR'
        nivel_riesgo = 'MODERADO'
    else:
        decision = 'APROBAR'
        nivel_riesgo = 'BAJO'

    return {
        'score_riesgo': round(proba, 4),
        'decision': decision,
        'nivel_riesgo': nivel_riesgo,
        'probabilidad_default': round(proba, 4),
        'umbral_usado': UMBRAL_RECHAZAR,
        'modelo': type(self.modelo).__name__,
    }

# Instancia global - singleton
predictor = Predictor()

```

**PASO  
4****api/app.py — Endpoints de la API**

api/app.py

```
python - api/app.py

"""
api/app.py - API REST del sistema de predicción de default crediticio.

Endpoints:
    POST /predecir -> score de riesgo del cliente
    GET  /health  -> estado de la API y metadatos del modelo
    GET  /        -> info básica
    GET  /docs     -> Swagger UI (autogenerado por FastAPI)

Ejecutar en Codespace:
    uvicorn api.app:app --host 0.0.0.0 --port 8000 --reload

Con Docker:
    docker compose up --build
"""

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware

from api.schemas import ClienteInput, PrediccionOutput, HealthResponse
from api.predictor import predictor

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | API | %(levelname)s | %(message)s',
    datefmt='%H:%M:%S',
)
log = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Carga el modelo al arrancar la API (startup event)."""
    log.info('Cargando modelo de riesgo crediticio...')
    predictor.cargar()
    log.info('API lista para recibir predicciones')
    yield
    log.info('API cerrando')

app = FastAPI(
    title='API de Riesgo Crediticio - Banco Wiese',
    description=(
        'Predicción de probabilidad de default crediticio. '
        'Modelo entrenado con el pipeline MLOps del proyecto Banco Wiese.'
    ),
    version='1.0.0',
    lifespan=lifespan,
)

# Permitir llamadas desde cualquier origen (CORS)
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_methods=['*'],
    allow_headers=['*'],
)
```

```
)

@app.get('/', tags=['Info'])
def root():
    """Información básica de la API."""
    return {
        'api': 'Banco Wiese - Predicción de Default Crediticio',
        'version': '1.0.0',
        'docs': '/docs',
        'health': '/health',
        'prediccion': 'POST /predecir',
    }

@app.get('/health', response_model=HealthResponse, tags=['Salud'])
def health():
    """Estado de la API y metadatos del modelo en producción."""
    if predictor.modelo is None:
        raise HTTPException(status_code=503, detail='Modelo no cargado')
    return HealthResponse(
        status='ok',
        modelo=type(predictor.modelo).__name__,
        version='1.0.0',
        recall_entrenamiento=predictor.mtricas.get('recall', 0.0),
    )

@app.post('/predecir', response_model=PrediccionOutput, tags=['Prediccion'])
def predecir(cliente: ClienteInput):
    """
    Predice la probabilidad de default de un cliente.

    - **score_riesgo**: probabilidad de default [0.0 - 1.0]
    - **decision**: APROBAR | REVISAR | RECHAZAR
    - **nivel_riesgo**: BAJO | MODERADO | ALTO | MUY ALTO
    """
    try:
        datos = cliente.model_dump()
        resultado = predictor.predecir(datos)
        log.info(
            'Predicción: score=%.4f decision=%s',
            resultado['score_riesgo'], resultado['decision'],
        )
        return PrediccionOutput(**resultado)
    except Exception as e:
        log.error('Error en predicción: %s', str(e))
        raise HTTPException(status_code=500, detail=str(e))
```

| Endpoint  | Método | Descripción                             |
|-----------|--------|-----------------------------------------|
| /predecir | POST   | Predicción de default para un cliente   |
| /health   | GET    | Estado de la API y metadatos del modelo |
| /docs     | GET    | Swagger UI autogenerado por FastAPI     |
| /         | GET    | Info básica y links de endpoints        |

**PASO  
5****Dockerfile — Imagen Docker de la API**

Archivo raíz

Cada instrucción crea una capa cacheada por Docker. Por eso `COPY requirements.txt` va antes que `COPY` del código: si solo cambia el código, Docker reutiliza la capa de `pip install`:

```
dockerfile — dockerfile

# Dockerfile — API de Riesgo Crediticio Banco Wiese

# — Imagen base: Python 3.10 slim (liviana, sin herramientas innecesarias) —
FROM python:3.10-slim

# — Metadatos —
LABEL maintainer="MLOps Banco Wiese"
LABEL description="API de predicción de default crediticio"
LABEL version="1.0.0"

# — Variables de entorno —
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=8000

# — Directorio de trabajo dentro del contenedor —
WORKDIR /app

# — Instalar dependencias del sistema (mínimas) —
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# — Copiar e instalar dependencias Python —
# IMPORTANTE: copiar requirements.txt ANTES del código
# Así Docker cachea esta capa y no reinstala si solo cambia el código
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# — Copiar el código de la API —
COPY api/ ./api/
COPY config.py .

# — Copiar artefactos del pipeline (modelo + features) —
COPY artifacts/modelo.pkl ./artifacts/
COPY artifacts/features.csv ./artifacts/
COPY artifacts/metrics.json ./artifacts/

# — Puerto que expone el contenedor —
EXPOSE 8000

# — Health check para que Docker sepa si el contenedor está sano —
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD python -c "import httpx; httpx.get('http://localhost:8000/health')"

# — Comando de inicio —
CMD ["uvicorn", "api.app:app", \
    "--host", "0.0.0.0", \
    "--port", "8000", \
    "--workers", "1"]
```

**Observación técnica — importante para poder replicar en casa:** el archivo en el repositorio se llama `dockerfile` (minúscula), pero `docker-compose.yml` hace referencia a `dockerfile: Dockerfile` (con D mayúscula). En Linux — el sistema operativo de GitHub Codespaces — los nombres de archivo distinguen mayúsculas de minúsculas, así que `docker compose up --build` fallará con «*no such file or directory*» tal como está. Antes de construir la imagen, renombra el archivo:

```
bash
```

```
# Ejecuta esto una sola vez, antes del PASO 7
mv dockerfile Dockerfile
```

```
text — .dockerignore
```

```
__pycache__/  
#*.pyc  
#*.pyo  
#.git/  
#.dvc/  
#mlruns/  
#notebooks/  
#data/  
#*.log  
#*.ipynb  
.env  
#artifacts/data_clean.csv  
#artifacts/X_test.csv  
#artifacts/y_test.csv  
#artifacts/reporte_evaluacion.png
```

## PASO 6

### docker-compose.yml — Infraestructura completa

Archivo raíz

Levanta la API + MLflow UI en un solo comando. En Codespaces, ambos puertos (8000 y 5000) se reenvían automáticamente:

```
yaml — docker-compose.yml
```

```
# docker-compose.yml — Levantar API + MLflow en un solo comando  
# Uso en Codespace: docker compose up --build  
  
version: '3.9'  
  
services:  
  
  # — Servicio 1: API de predicción —————  
  api:  
    build:  
      context: .  
      dockerfile: Dockerfile  
    container_name: banco-wiesse-api  
    ports:  
      - '8000:8000'      # Codespace reenvía este puerto automáticamente  
    volumes:  
      - ./artifacts:/app/artifacts  # montar artefactos del pipeline  
    environment:  
      - PIPELINE_VERSION=v1.0.0  
    restart: unless-stopped
```

```

healthcheck:
  test: ['CMD', 'python', '-c',
        'import httpx; httpx.get("http://localhost:8000/health")']
  interval: 30s
  timeout: 10s
  retries: 3
networks:
  - banco-wiesse-net

# — Servicio 2: MLflow UI (tracking de experimentos) —————
mlflow:
  image: python:3.10-slim
  container_name: banco-wiesse-mlflow
  command: >
    bash -c 'pip install mlflow -q &&
              mlflow ui --host 0.0.0.0 --port 5000
              --backend-store-uri file:///mlruns'
  ports:
    - '5000:5000'      # Codespace reenvía este puerto automáticamente
  volumes:
    - ./mlruns:/mlruns
  networks:
    - banco-wiesse-net

networks:
  banco-wiesse-net:
    driver: bridge

```

PASO  
7

## Construir la imagen y ejecutar el contenedor

Terminal

En GitHub Codespaces, Docker ya está disponible en la terminal integrada — no necesitas instalar Docker Desktop.

bash — Opción A — Docker directo

```
cd /workspaces/banco-wiesse-mlops
```

# 1. Construir la imagen (usa el Dockerfile renombrado en el Paso 5)

```
docker build -t banco-wiesse-api:v1.0 .
```

# 2. Ejecutar el contenedor

```
docker run -d --name banco-wiesse-api -p 8000:8000 \
  -v $(pwd)/artifacts:/app/artifacts \
  banco-wiesse-api:v1.0
```

# 3. Verificar que está corriendo

```
docker ps
```

# 4. Ver los logs de inicio

```
docker logs banco-wiesse-api
```

bash — Opción B — Docker Compose (recomendada)

```
docker compose up --build
```

```
# Container banco-wiesse-api      Started -> puerto 8000
```

```
# Container banco-wiesse-mlflow   Started -> puerto 5000
```

```
# Para detener todos los servicios:  
docker compose down
```

■ En Codespaces: ve a la pestaña **PORTS** y haz clic en el globo del puerto 8000 para la API, o del puerto 5000 para MLflow UI.

**PASO  
8****Testing manual del endpoint /predecir**

Terminal

```
bash – TEST 1 – Verificar que la API está viva
```

```
curl http://localhost:8000/health  
# {"status":"ok","modelo":"RandomForestClassifier",  
#  "version":"1.0.0","recall_entrenamiento":0.4923}
```

```
bash – TEST 2 – Cliente de ALTO RIESGO
```

```
curl -X POST http://localhost:8000/predecir \  
-H 'Content-Type: application/json' \  
-d '{  
  "Prct_uso_tc": 0.92, "Prct_deuda_vs_ingresos": 0.78,  
  "Mto_ingreso_mensual": 1200.0, "Nro_dependiente": 3.0,  
  "Edad": 28, "Nro_prestao_retrasados": 5,  
  "Nro_prod_financieros_deuda": 10, "Nro_retraso_60dias": 3,  
  "Nro_creditos_hipotecarios": 0, "Nro_retraso_ult3anios": 4  
}'  
# {"score_riesgo":0.87,"decision":"RECHAZAR","nivel_riesgo":"MUY ALTO",...}
```

```
bash – TEST 3 – Validación Pydantic (HTTP 422)
```

```
curl -X POST http://localhost:8000/predecir \  
-H 'Content-Type: application/json' -d '{"Prct_uso_tc": -999}'  
# HTTP 422 Unprocessable Entity
```

**Swagger UI automático en /docs:** FastAPI genera una interfaz interactiva en `/docs`. Abre la URL pública del puerto 8000 y agrega `/docs` al final.

**PASO  
9****tests/test\_api.py — Tests automáticos**

tests/

No requiere Docker corriendo — usa el `TestClient` de FastAPI, que levanta la app en memoria y ejecuta el `lifespan` (carga el modelo real):

```
python – tests/test_api.py  
  
"""  
tests/test_api.py – Tests del endpoint /predecir con TestClient de FastAPI.  
  
No requiere Docker corriendo. Usa TestClient que levanta la app en memoria.
```

```

Ejecutar: pytest tests/test_api.py -v

NOTA: Se usa el fixture `client` con scope="session" y context manager
para que el lifespan (startup) se ejecute y el modelo se cargue correctamente.
"""

import pytest
from fastapi.testclient import TestClient

import sys
from pathlib import Path
sys.path.append(str(Path(__file__).resolve().parents[1]))

from api.app import app

# — Fixture —

@pytest.fixture(scope="session")
def client():
    """Levanta la app completa con lifespan (carga el modelo en startup)."""
    with TestClient(app) as c:
        yield c

# — Datos de prueba —

CLIENTE_ALTO_RIESGO = {
    'Prct_uso_tc': 0.92,
    'Prct_deuda_vs_ingresos': 0.78,
    'Mto_ingreso_mensual': 1200.0,
    'Nro_dependiente': 3.0,
    'Edad': 28,
    'Nro_prestao_retrasados': 5,
    'Nro_prod_financieros_deuda': 10,
    'Nro_retraso_60dias': 3,
    'Nro_creditos_hipotecarios': 0,
    'Nro_retraso_ultim3anios': 4,
}

CLIENTE_BAJO_RIESGO = {
    'Prct_uso_tc': 0.15,
    'Prct_deuda_vs_ingresos': 0.12,
    'Mto_ingreso_mensual': 5500.0,
    'Nro_dependiente': 1.0,
    'Edad': 45,
    'Nro_prestao_retrasados': 0,
    'Nro_prod_financieros_deuda': 3,
    'Nro_retraso_60dias': 0,
    'Nro_creditos_hipotecarios': 2,
    'Nro_retraso_ultim3anios': 0,
}

# — Helpers —

DECISIONES_VALIDAS = {'APROBAR', 'REVISAR', 'RECHAZAR'}
NIVELES_RIESGO_VALIDOS = {'BAJO', 'MODERADO', 'ALTO', 'MUY ALTO'}

def assert_respuesta_valida(body: dict):
    """Verifica que la estructura y tipos de la respuesta sean correctos."""
    assert 'score_riesgo' in body
    assert 'decision' in body
    assert 'nivel_riesgo' in body
    assert 'probabilidad_default' in body
    assert 'umbral_usado' in body
    assert 'modelo' in body
    assert 0.0 <= body['score_riesgo'] <= 1.0
    assert body['decision'] in DECISIONES_VALIDAS
    assert body['nivel_riesgo'] in NIVELES_RIESGO_VALIDOS

```

```
# Consistencia interna: score y decision deben ser coherentes
score = body['score_riesgo']
umbral = body['umbral_usado']
if score >= umbral:
    assert body['decision'] == 'RECHAZAR'
elif score >= 0.40:
    assert body['decision'] == 'REVISAR'
else:
    assert body['decision'] == 'APROBAR'

# — Tests —

def test_health_ok(client):
    """El endpoint /health debe retornar status ok."""
    r = client.get('/health')
    assert r.status_code == 200
    data = r.json()
    assert data['status'] == 'ok'
    assert 'modelo' in data
    assert 'recall_entrenamiento' in data
    assert isinstance(data['recall_entrenamiento'], float)

def test_root_info(client):
    """El endpoint raíz debe retornar info básica de la API."""
    r = client.get('/')
    assert r.status_code == 200
    data = r.json()
    assert 'api' in data
    assert 'docs' in data

def test_predecir_alto_riesgo_estructura(client):
    """POST /predecir debe retornar 200 con estructura correcta."""
    r = client.post('/predecir', json=CLIENTE_ALTO_RIESGO)
    assert r.status_code == 200
    assert_respuesta_valida(r.json())

def test_predecir_bajo_riesgo_estructura(client):
    """POST /predecir debe retornar 200 con estructura correcta."""
    r = client.post('/predecir', json=CLIENTE_BAJO_RIESGO)
    assert r.status_code == 200
    assert_respuesta_valida(r.json())

def test_decision_coherente_con_score(client):
    """La decisión debe ser siempre coherente con el score y el umbral."""
    for payload in [CLIENTE_ALTO_RIESGO, CLIENTE_BAJO_RIESGO]:
        r = client.post('/predecir', json=payload)
        assert r.status_code == 200
        assert_respuesta_valida(r.json())

def test_score_es_determinista(client):
    """El mismo cliente debe recibir siempre el mismo score."""
    r1 = client.post('/predecir', json=CLIENTE_ALTO_RIESGO)
    r2 = client.post('/predecir', json=CLIENTE_ALTO_RIESGO)
    assert r1.status_code == 200
    assert r2.status_code == 200
    assert r1.json()['score_riesgo'] == r2.json()['score_riesgo']

def test_validacion_campos_invalidos(client):
    """Campos inválidos deben retornar HTTP 422 Unprocessable Entity."""
    r = client.post('/predecir', json={'Prct_uso_tc': -999})
```

```
assert r.status_code == 422

def test_validacion_edad_menor_18(client):
    """Edad menor a 18 debe retornar HTTP 422."""
    datos = CLIENTE_BAJO_RIESGO.copy()
    datos['Edad'] = 15
    r = client.post('/predecir', json=datos)
    assert r.status_code == 422

def test_validacion_ingreso_negativo(client):
    """Ingreso mensual negativo debe retornar HTTP 422."""
    datos = CLIENTE_BAJO_RIESGO.copy()
    datos['Mto_ingreso_mensual'] = -500.0
    r = client.post('/predecir', json=datos)
    assert r.status_code == 422

def test_swagger_ui_disponible(client):
    """La documentación Swagger UI debe estar disponible en /docs."""
    r = client.get('/docs')
    assert r.status_code == 200

def test_openapi_schema(client):
    """El schema OpenAPI debe estar disponible en /openapi.json."""
    r = client.get('/openapi.json')
    assert r.status_code == 200
    schema = r.json()
    assert 'paths' in schema
    assert '/predecir' in schema['paths']
    assert '/health' in schema['paths']
```

```
bash
```

```
cd /workspaces/banco-wiesse-mlops
pytest tests/test_api.py -v
```

**PASO  
10****Commit final desde el Codespace**

Git

```
bash
```

```
cd /workspaces/banco-wiesse-mlops
git add api/ Dockerfile docker-compose.yml .dockerignore tests/ requirements.txt
git commit -m "feat: API FastAPI + Docker para servir modelo crediticio"
git push origin main
git log --oneline
```

## Referencia rápida de comandos Docker

| Comando                                            | Para qué sirve                         |
|----------------------------------------------------|----------------------------------------|
| <code>docker build -t nombre:tag .</code>          | Construir imagen desde el Dockerfile   |
| <code>docker run -d -p 8000:8000 nombre:tag</code> | Ejecutar contenedor en background      |
| <code>docker ps</code>                             | Ver contenedores en ejecución          |
| <code>docker logs &lt;container&gt;</code>         | Ver logs del contenedor                |
| <code>docker compose up --build</code>             | Levantar todos los servicios definidos |
| <code>docker compose down</code>                   | Detener y eliminar contenedores        |
| <code>pytest tests/test_api.py -v</code>           | Ejecutar tests automáticos, sin Docker |

## ¿Qué aprendiste en este tema?

### 1. Qué es Docker y su rol en MLOps

Un contenedor empaqueta el modelo, el código y las dependencias. Garantiza que la API funcione igual en desarrollo, Codespace y en los servidores del banco.

### 2. Creación de una imagen con dependencias ML

El Dockerfile define la imagen capa por capa. Copiar requirements.txt antes que el código activa el cache de Docker.

### 3. FastAPI para servir modelos en producción

`app.py` define los endpoints, `schemas.py` valida con Pydantic, y el modelo se carga una sola vez en el startup (`lifespan`).

### 4. Testing manual y automático del endpoint

curl y Swagger UI para pruebas manuales; TestClient de FastAPI para tests automáticos sin levantar Docker.

### 5. Reproducibilidad de la infraestructura

`docker compose up --build` levanta API + MLflow en un solo comando — cualquier persona con acceso al Codespace puede probar la API.

**En el Tema 6...** Optimizaremos hiperparámetros y registraremos el modelo en el Model Registry de MLflow para gestionar versiones en producción.

## Checklist final

- ☐ Verifiqué que `artifacts/modelo.pkl` existe (Tema 4 completado).
- ☐ Escribí `api/schemas.py`, `api/predictor.py` y `api/app.py`.
- ☐ Renomé `dockerfile` a `Dockerfile` antes de construir la imagen.

- ☐ Construí la imagen y levanté el contenedor (o usé `docker compose up --build`).
- ☐ Probé `/health`, `/predecir` y `/docs` con curl y el navegador.
- ☐ `pytest tests/test_api.py -v` pasa todos los tests.
- ☐ Hice commit + push final y lo verifiqué en github.com.